Aalto University
School of Science

Workflows for Big Data Platforms

Hong-Linh Truong
Department of Computer Science
linh.truong@aalto.fi

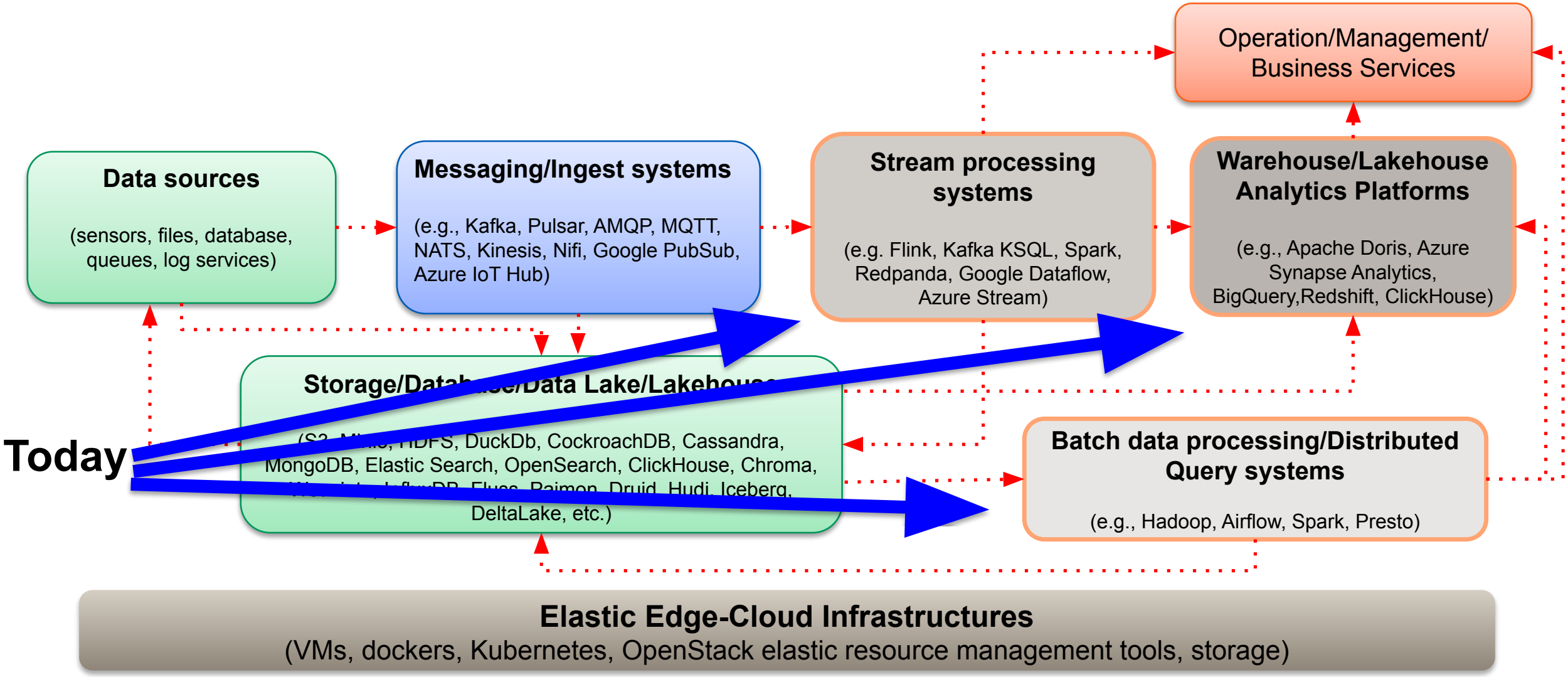
25.02.2026

Content is available under
CC BY-SA 4.0 unless otherwise stated

Learning objectives

- Understand the role and use cases of workflows in big data platforms
- Understand key concepts and techniques in workflows and able to design workflows
- Able to design workflows for basic data platform tasks
- Able to apply common workflow technologies for practical work

Our big data at large-scale: the big picture in this course



Complex use cases need diverse types of capabilities

- On-demand deployment and configuration
  - prepare execution environments and platform components
  - enable compute resource elasticity needs
- Data ingestion, lifecycle management (data cleansing and backup)
  - access and coordinate many different compute services, data sources, ingestion and extraction applications
- Analytics and AI/ML support
  - orchestrate AI/ML pipelines and communication with humans/optimization services
  - perform metrics understanding, user activities analytics, and customer understanding

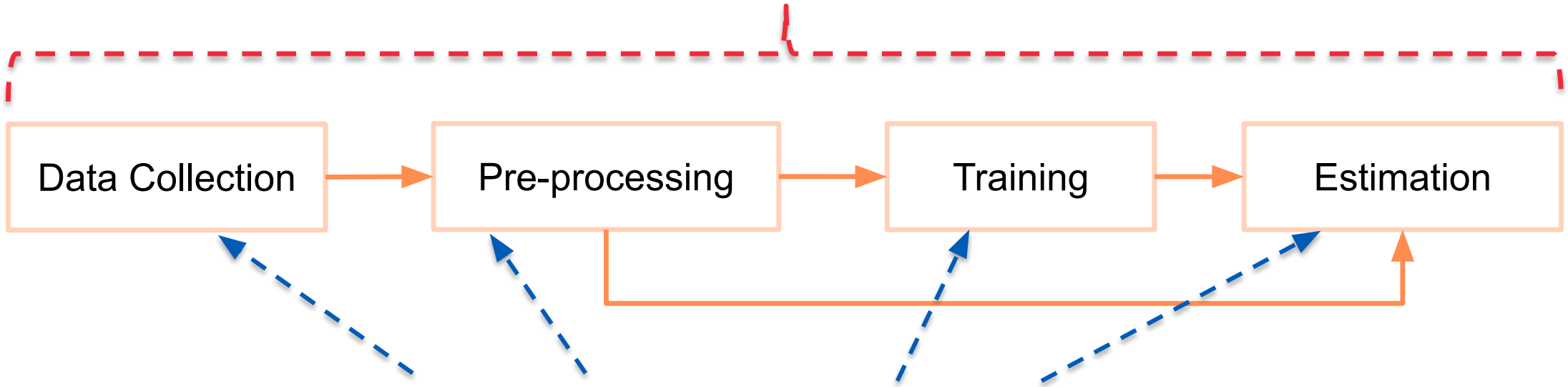
Tasks in big data platforms

- Loosely coupled data processing related tasks
  - data collection and transformation
    - data transfers, extraction, transformation,
  - data processing, including machine learning
    - data analytics, training, serving machine learning algorithms
  - across/involvement of multiple frameworks
- Loosely coupled platform automation tasks
  - service deployment, resource elasticity, and backup/recovery
- Business tasks integration with big data analytics
  - integration with customer services, bringing insights from data analytics to business decision making

Tasks for data analysis and AI/ML

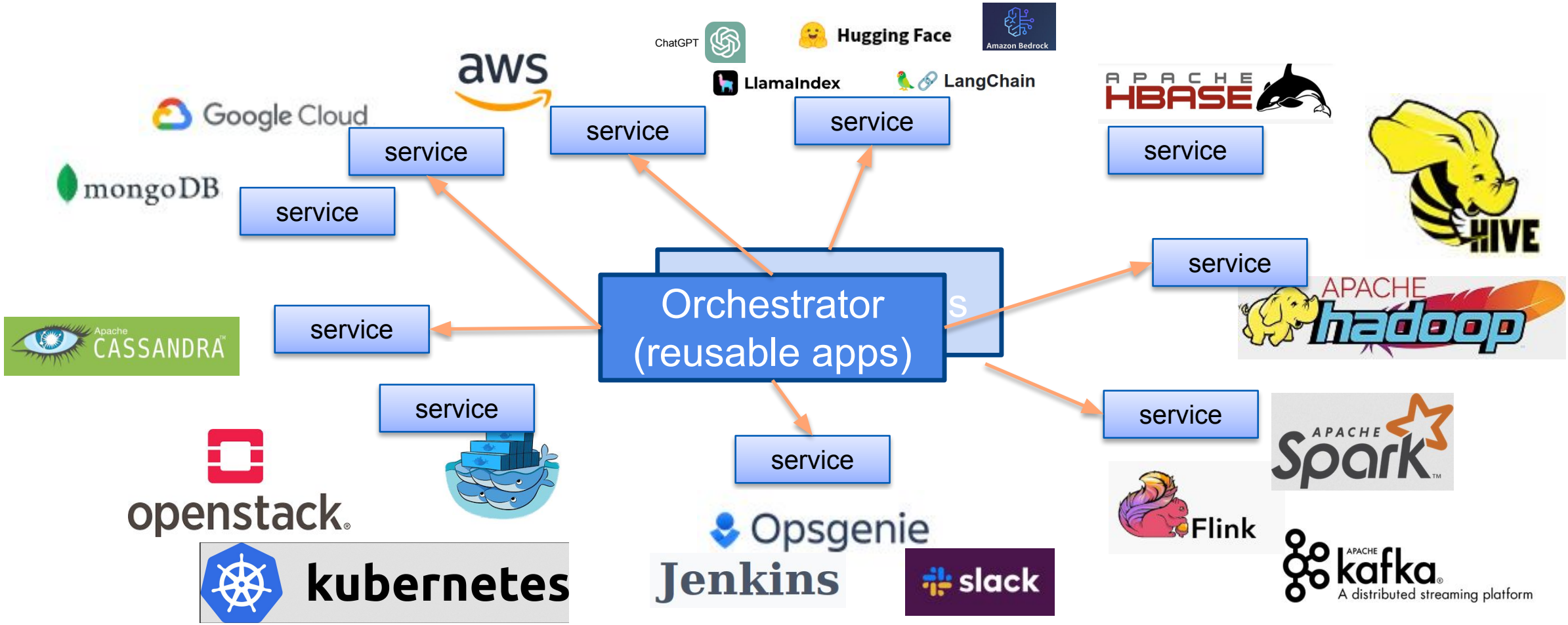
Multiple phases of analysis and training using different tools/platforms

(Meta) orchestration across sub phases/systems



Orchestration: within tasks in a phase, for different tools (e.g., Apache Spark, MLflow, Ray)

Service capabilities in data platforms: more than just with “data processing”



Workflows

- A workflow specifies a process
  - consists of a set of connected tasks/activities
  - has tasks/activities carried out by diverse types of software services or humans, each performs a function
  - can be automated with/without human intervention
  - has data/control task dependencies
  - can be reusable (tasks, part of the workflow, and the whole workflow)

Workflow technologies

- Given many services offering different capabilities, we can combine them for different cases
  - orchestration of capabilities from different services as the key!
  - reuse/customization of capabilities with a given set of services
- Workflows are flexibly defined and changed
  - individual services cannot be changed easily
  - but there are many ways to combine such services!
  - the integration is loosely coupled

We have many workflows that are built in a flexible way for different goals

How to build the workflows and orchestrate tasks in these workflows?

Tasks and workflows

- Diverse types of tasks
  - task can be simple or complex (e.g., a task running an AI inference)
  - tasks are performed by software and humans
  - including IoT devices, robots, LLMs and AI Agents
- Workflow
  - coordinate/orchestrate many tasks:
    - *the function of tasks **is not** really "carried out/executed" by workflows* ⇒ *orchestration/coordination*
  - workflow can be simple, like a pipeline of a sequence of tasks or complex with many forks/loops of tasks

Workflow and pipeline/data workflow

- Data workflow ⇒ data pipeline
  - *“a pipeline is a set of data processing elements connected in series, where the output of one element is the input of the next one”*
    source: https://en.wikipedia.org/wiki/Pipeline_%28computing%29
- Two interpretations in practice:
  - a pipeline is a workflow with a simple structure
  - a pipeline coordinates different (sub)workflows
- Note: sometimes the “data pipeline” here is just an abstract design
  - a high-level design view, not an implementation

Stick to the term: **workflow**

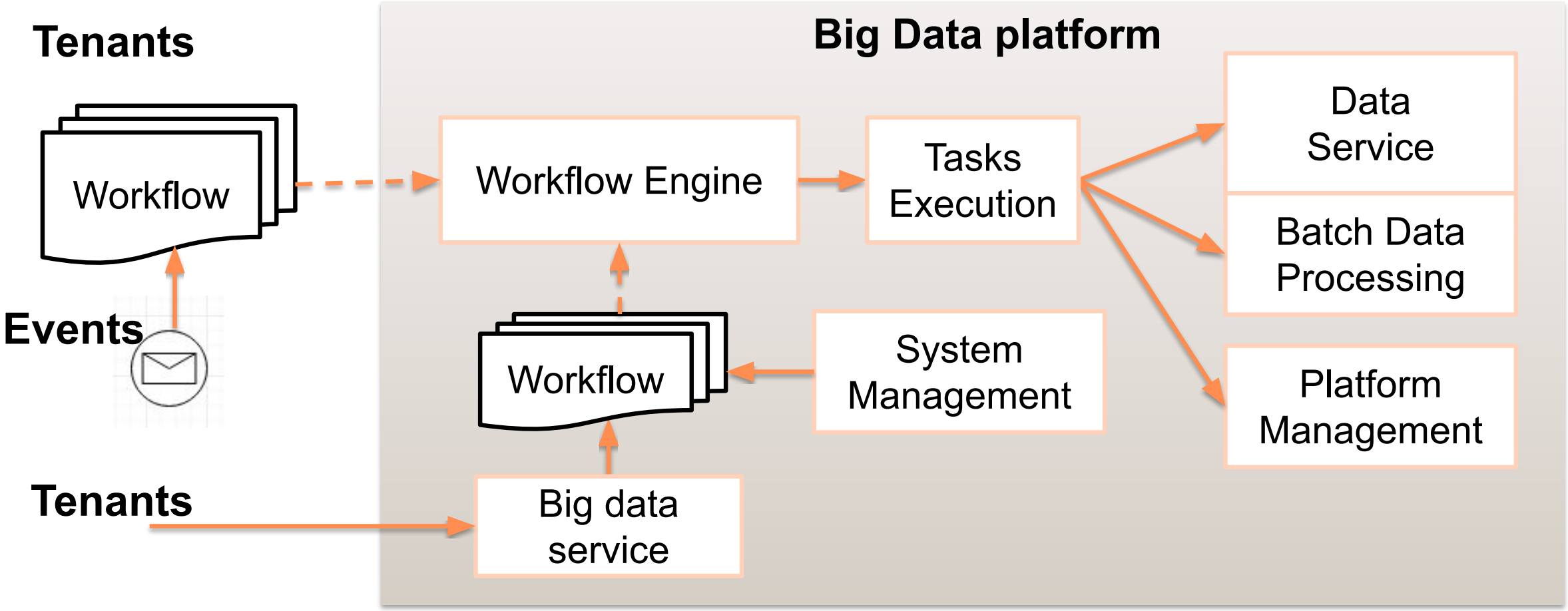
Types of workflows

- Data workflows
  - mainly for data processing/analysis in various domains
- Business workflows/processes
  - business processes in enterprise computing (e.g., BI, ERP, and e-commerce)
- Scientific workflows
  - in scientific computing and high performance computing (e.g., bioinformatics, Next generation sequencing, astrophysics, material science simulations)
- DevOps workflows
  - at system level for automating infrastructure provisioning, system recovery, software testing, etc.
- AI/ML workflows
  - AI/ML training and inferences

Key components

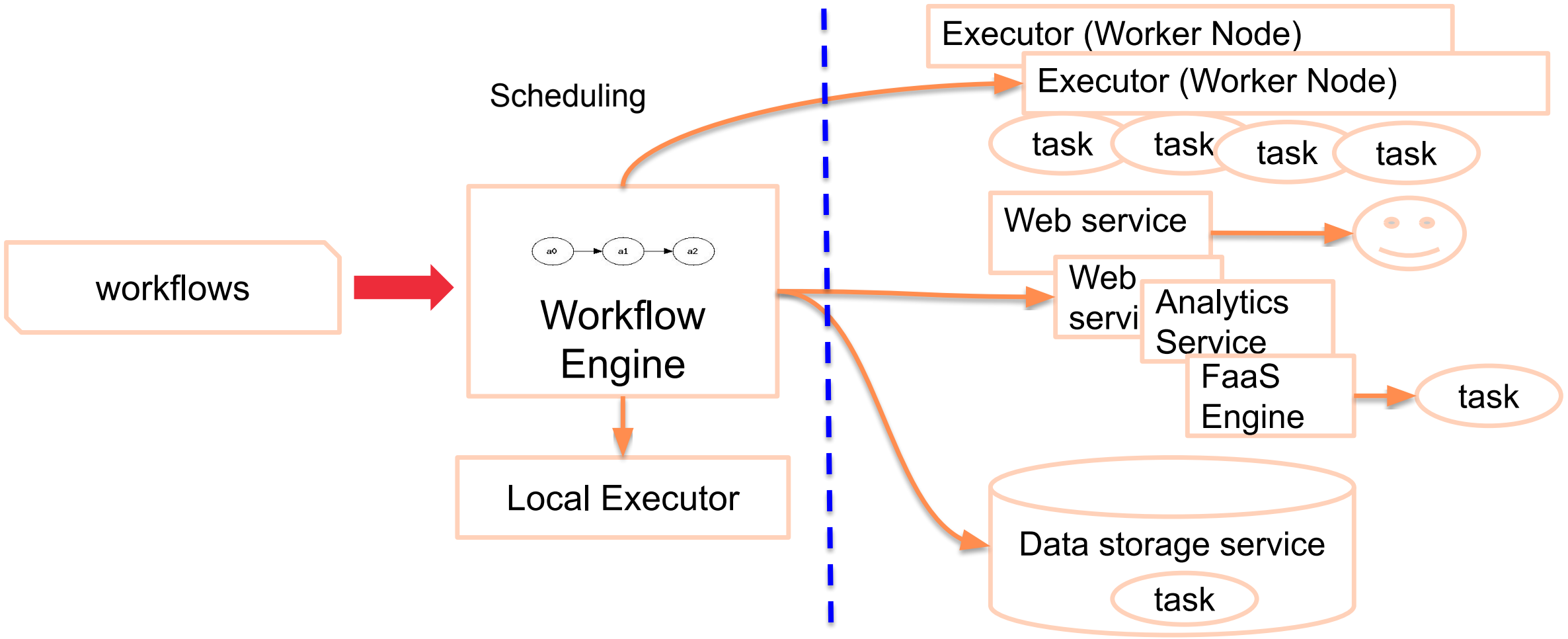
- Tasks/activities
  - describe a single work (it does not mean small)
  - tasks can be carried out by humans, executables, scripts, batch applications, stream applications and other types of services
- Workflow languages
  - structure/describe tasks, dataflows, and control flows
- Workflow engines
  - execute the workflow by orchestrating tasks
  - usually *call remote services* to run tasks

Workflows in big data platforms



Common workflow execution models

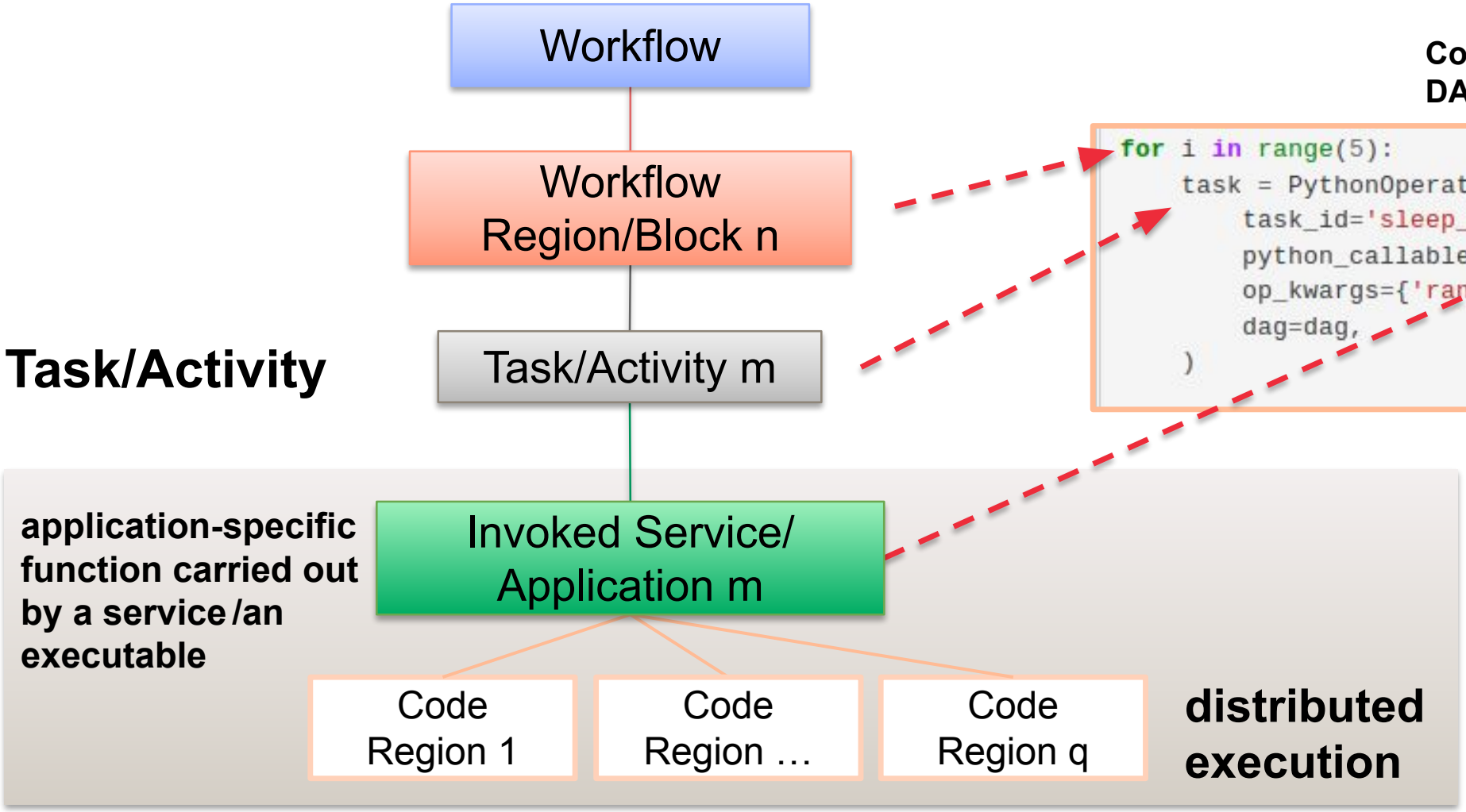
Major works are carried out in distributed nodes (manage by Kubernetes, Celery, Dask, Ray, Slurm,…)



Select/build workflows in a platform

- Rich data services
  - for data storing/retrieving tasks
- Big data computation engines
  - for data processing tasks with different workloads: ML and (batch/stream) big data processing
- Different underlying cloud/distributed computing infrastructures
  - for resource management tasks and workflow infrastructures
- REST APIs and messaging systems integration
  - for widely integration with other services (e.g., business services)

Structured view of workflows



Code captured from Airflow:
DAG: example_python_operator

```python
for i in range(5):
    task = PythonOperator(
        task_id='sleep_for_' + str(i),
        python_callable=my_sleeping_function,
        op_kwargs={'random_base': float(i) / 10},
        dag=dag,
    )
```

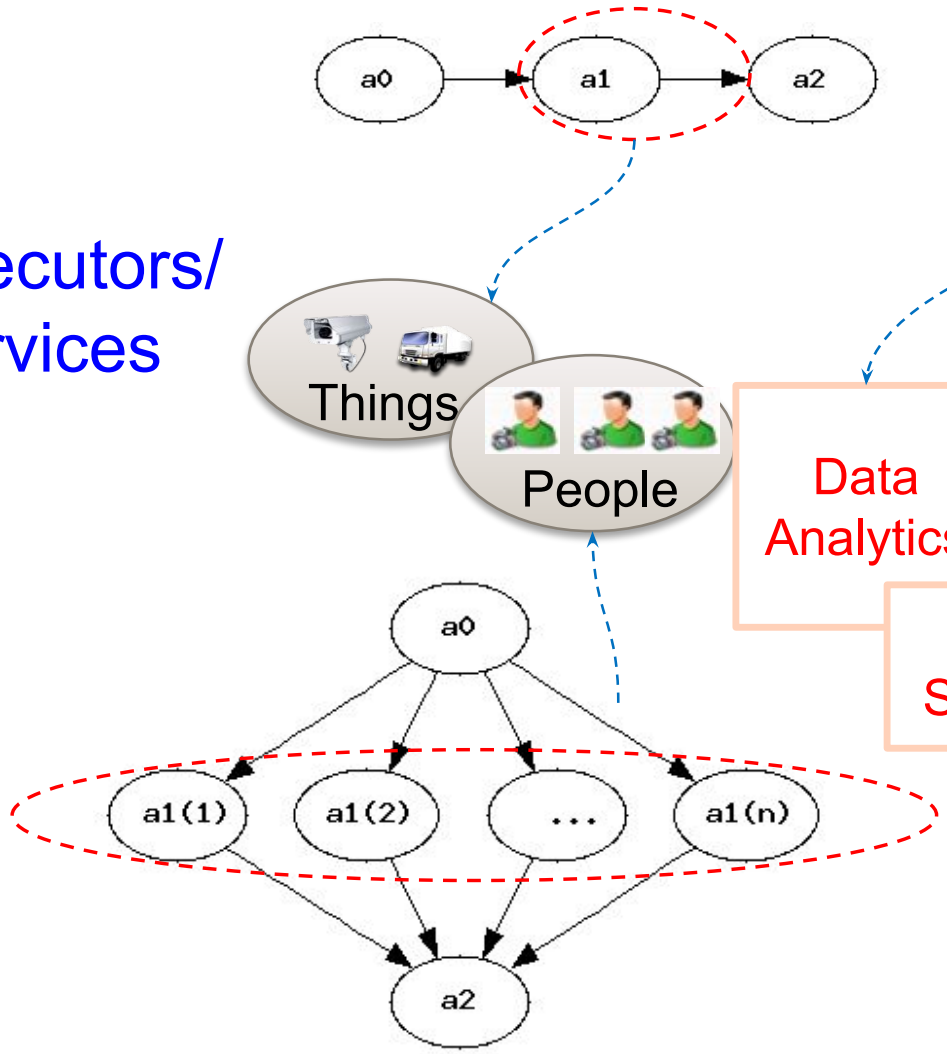
Note: the invoked service/application can be complex (e.g., another workflow)

Describing workflows

- Programming languages with procedural code + declarative schemas
  - general- and specific-purpose programming languages, such as Java, Python, Swift + YAML/JSON (declarative configurations)
  - common ways in big data platforms for data analytics and system automation
  - data processing workflows: common in data science and data analysis
- Descriptive languages, declarative schemas and code
  - BPEL, YAML, JSON-based specifications and several languages designed for specific workflow engines
  - common in business and scientific workflows

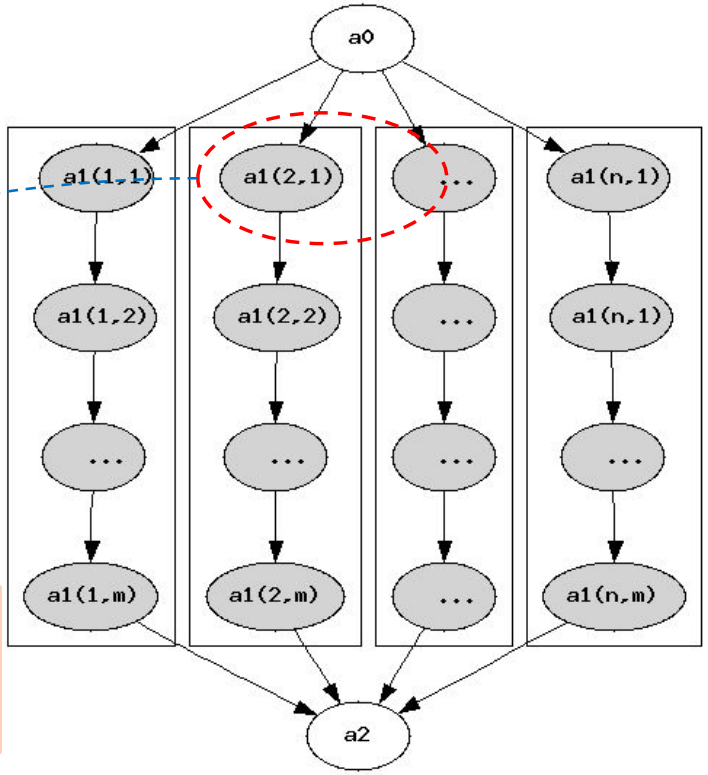
Task dependencies & orchestration

Executors/
Services



Data
Analytics

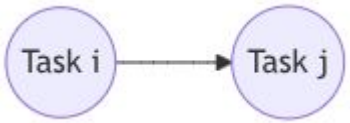
AI/ML/LLM
Service/Agent

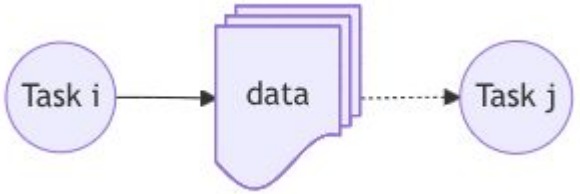


Structures of the workflow: DAG (Directed Acyclic Graph) is a common model

Control flow vs data flow

- Control flow - orchestrating tasks
  - Explicit control dependencies among tasks
    - e.g., Airflow
  - data exchange space between tasks: used and managed by tasks (the developer)
- Data dependencies – orchestrating data
  - a task is triggered based on the availability of the data it depends
    - e.g., Dagster (https://dagster.io/)
  - data exchange space between tasks: managed by the workflow system via IO management



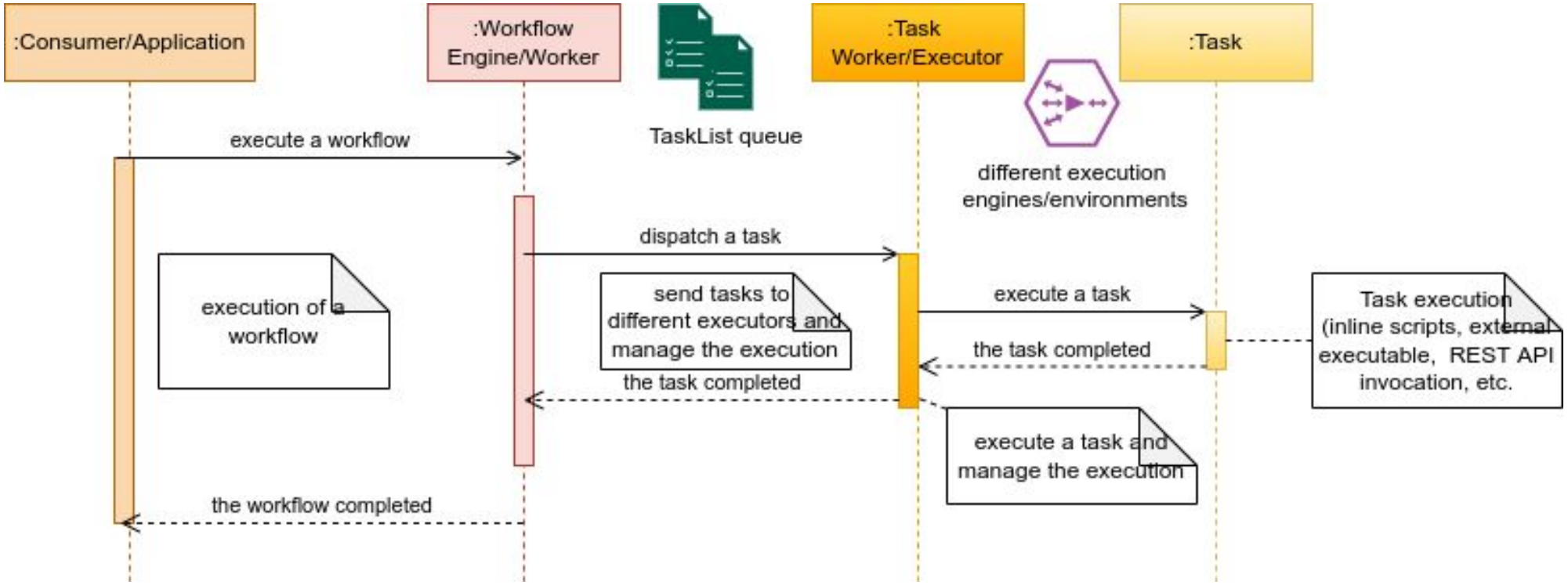


implicit vs explicit
lightweight vs heavy
ephemeral vs persistent

Modern data workflows in clouds

- Invoked services/applications
  - cloud services: with diverse types of APIs and protocols
  - serverless functions
  - AI/ML/LLM agents/services
- Workers/Executors
  - distributed workers in distributed compute nodes of data centers
    - interfaces via APIs and messaging systems
  - containers and distributed task executors
  - cloud orchestration
- Python-based for data analytics and ML workflows
  - mainly for data transformation and analysis

Complex execution model across distributed machines



Challenges in managing dependencies, failures and performance

Subworkflow

- A subworkflow consists of many tasks
  - structured like a workflow
  - called by the main workflow

    ⇒ functionality split
- Why
  - reuse for common purposes
  - maintainability and composition
  - dynamic execution and scaling

Airflow Taskgroup - Subworkflow

```python
@task
def download_data_task(source_file, dest_file):
    download_data(source_file, dest_file)

@task
def aggregation_task(input_file, report_destination):
    basic_aggregation(input_file, report_destination)

@task_group(group_id="download_and_analytics")
def subworkflow_analytics(source_file, temp_dest_file, report_destination):
    d = download_data_task(source_file, temp_dest_file)
    a = aggregation_task(temp_dest_file, report_destination)
    d >> a
    return a
```

Flyte Subworkflow

```python
@fl.workflow
def sub_workflow_1():
    t1()


@fl.workflow
def sub_workflow_0():
    t0()


@fl.workflow
def chain_workflows_wf():
    sub_wf1 = sub_workflow_1()
    sub_wf0 = sub_workflow_0()

    sub_wf0 >> sub_wf1
```

Code source:
https://www.union.ai/docs/v1/flyte/user-guide/programming/chaining-entities/

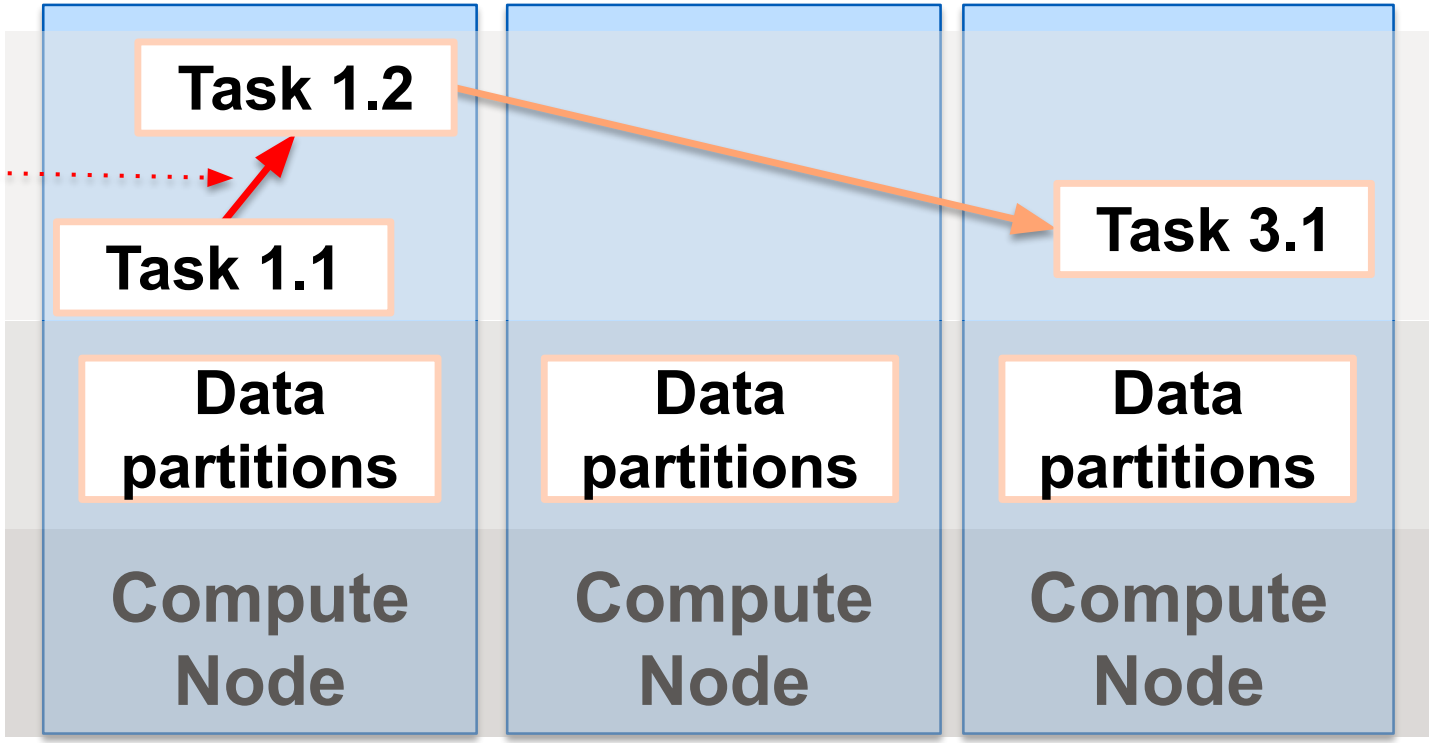
Runtime aspects

- Parallel and distributed execution
  - tasks are executed in different machines (by external invoked applications/services) ⇒ multiple running workflows in the same system
- Long and/or periodic running
  - can be hours or weeks! ⇒ pausing and resuming workflows are normal
- Checkpoint and recovery
  - dealing with failures at different levels: workflows and tasks retry/recovery
- Monitoring and tracking
  - States and performance metrics: queuing, running, idle, suspended, failed
- Stateful management
  - dependencies among tasks w.r.t control and data, stateful tasks ⇒ global services for managing states and data among tasks

Recall: exchange data possibilities

Try to analyze and identify this problem with a framework of your choice (and fault management)

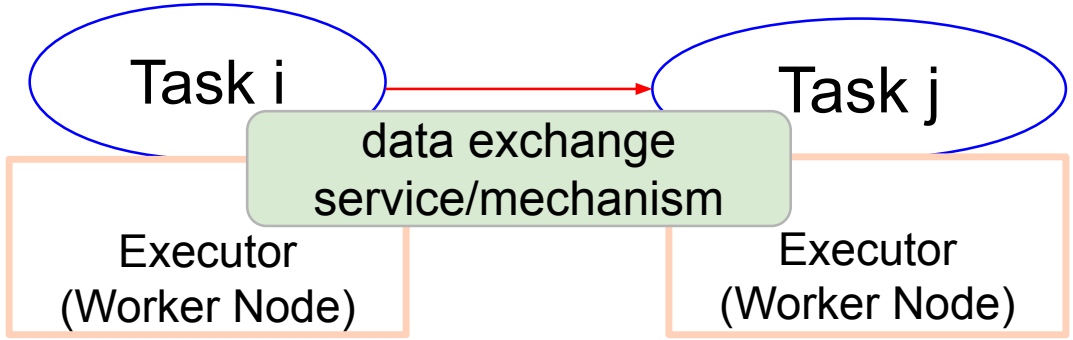
Need to exchange data



State and exchange data

- How do tasks exchange data, when a task does not know another task?

By **value or by reference** exchange?



Possibilities - multi-tiering:

- in-memory
- local
- shared file systems
- global/common storage systems

Pull/State-check

- key/value database
- external processes for pulling data

Some important aspects:

- Asynchronous task execution in which the task results must be retrieved with a different method
- Idempotency (retries, fault-tolerance) and caching
- SerDe (serialization and deserialization)
- Performance of shared systems/services for data exchange
- Deployment consideration

Scheduling and resource management (1)

- Scheduling
  - scheduling in a large resource pool, e.g., using clusters of compute nodes
  - workflow- and task-level scheduling
- Elasticity
  - elasticity controls of virtualized resources (workers/executors, e.g. VMs/containers/Kubernetes) for executing tasks
- Multiple levels of concurrency/parallelism
  - cluster level vs node level

- Examples
  - periodic cron schedules, backfill, opportunistic schedules
  - increase number of distributed workers/cluster sizes
  - heterogeneous resources for tasks: lightweight compute nodes & high-end nodes

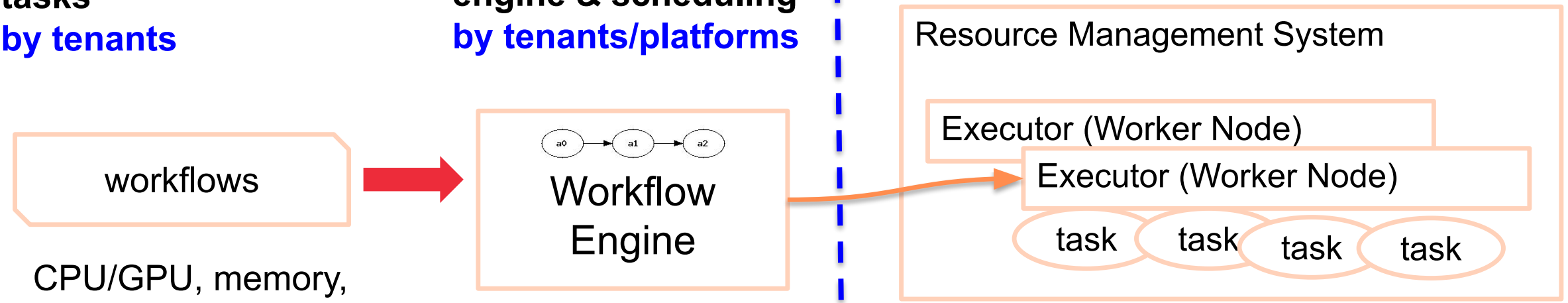
Wu, F., Wu, Q. & Tan, Y. Workflow scheduling in cloud: a survey. J Supercomput 71, 3373–3418 (2015). https://doi.org/10.1007/s11227-015-1438-4
Mainak Adhikari, Tarachand Amgoth, and Satish Narayana Srirama. 2019. A Survey on Scheduling Strategies for Workflows in Cloud Environment and Emerging Trends. ACM Comput. Surv. 52, 4, Article 68 (August 2019), 36 pages. https://doi.org/10.1145/3325097

Scheduling and resource management (2)

Resource specified for individual workflows or tasks
by tenants

Resource management at the engine & scheduling
by tenants/platforms

Resource consumption & management at executors by platforms (when resources are not own by tenants)



CPU/GPU, memory, storage, e.g., using decorators or config parameters

e.g., https://docs.dagster.io/deployment/oss/deployment-options/kubernetes/customizing-your-deployment

max concurrent workflow executions, max parallel tasks, workers, etc.

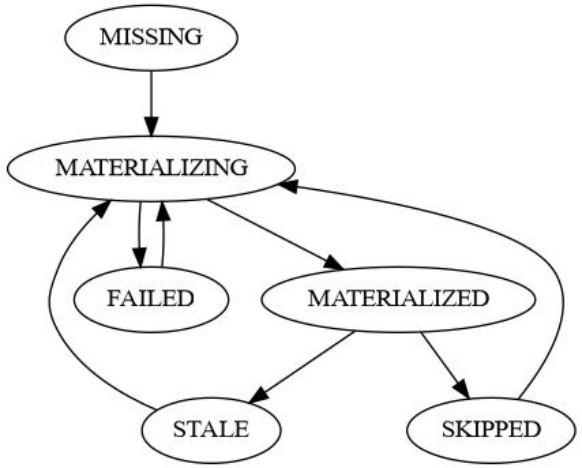
CPU/GPU, memory, usage time, etc. via configurations of executors (e.g., Kubernetes, Celery, & Slurm)

Discussion: the need of service level agreements between tenants and platforms w.r.t workflow execution?

Monitoring

- Understand the states of tasks and the whole workflow
  - workflow level vs task/activity level vs invoked service/application level
- States of assets/data dependencies
- Multiple levels of monitoring
  - Workflow Engine/Worker
  - Scheduler
  - Task worker/executor
  - External services/applications

Example of possible states of assets in Dagster (https://dagster.io/)



Possible stages and states of tasks

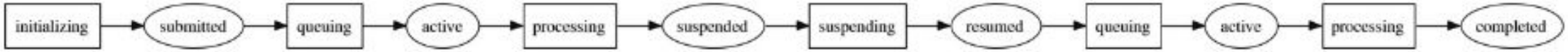
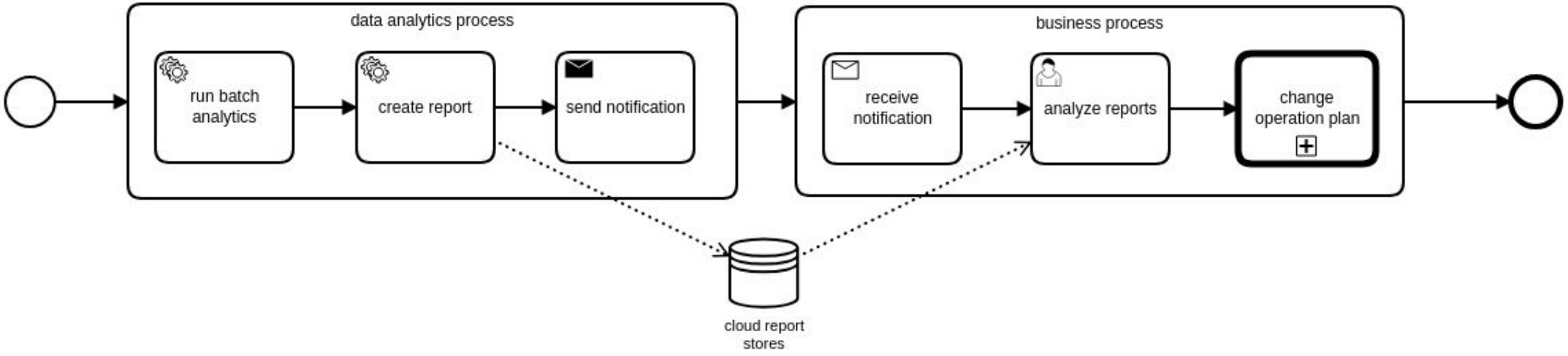


Fig. 3. Discrete process model of the tracing execution of an activity. □ represents an execution phase, ◯ represents an event.

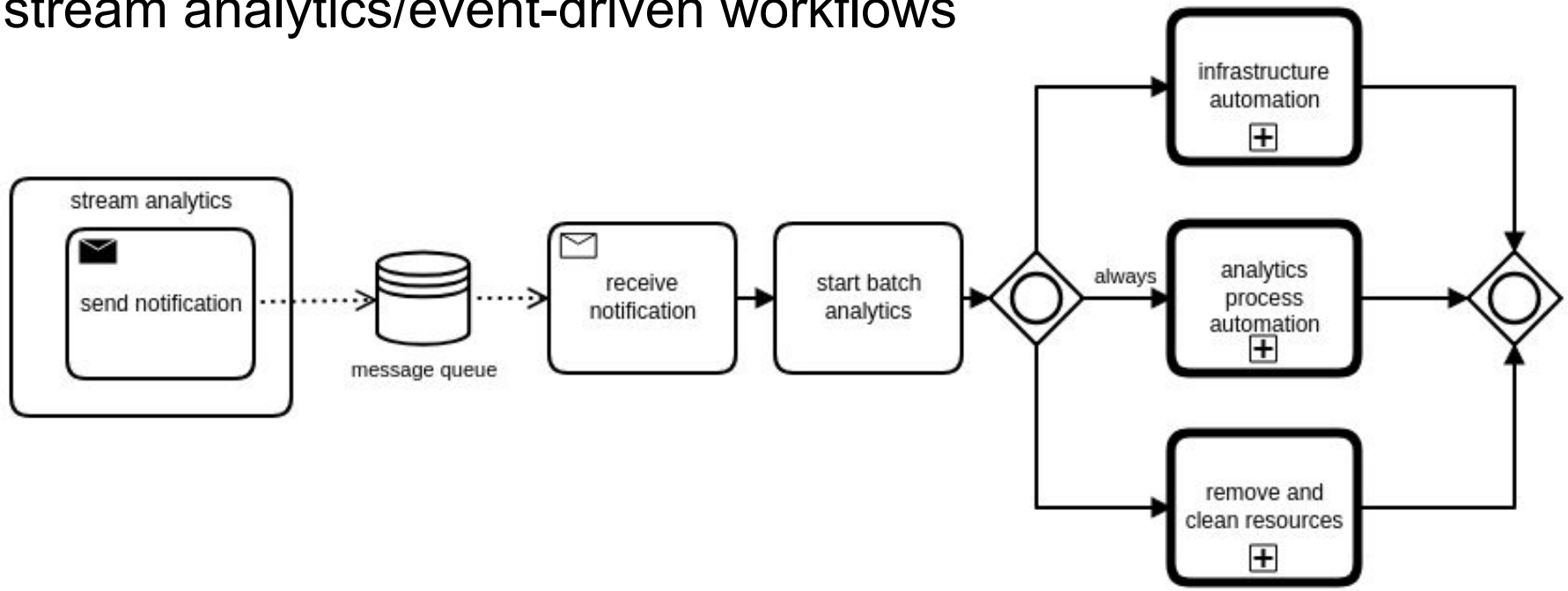
Integrating workflows for different purposes (1)

- Integration
  - data analytics processes and business processes
  - include human-in-the-loop



Integrating workflows for different purposes (2)

- Integration
  - multiple types of workflows for services/infrastructure provisioning and analytics
  - stream analytics/event-driven workflows



Existing frameworks for practical study

- **Apache Oozie**: designed to work with Hadoop
  - orchestrating Hadoop jobs
- **Serverless-based**: Function-as-a-Service
  - e.g., Microsoft, Google, AWS serverless/function-as-a-service, https://www.union.ai/, https://serverlessworkflow.io/
- **Apache Airflow**: a generic workflow framework (hands-on)
- **Argo Workflows**: container-native workflow engine
- **Uber Cadence** (https://cadenceworkflow.io) & **Camunda** (https://camunda.com/): connecting to business activities + human in the loop
- **N8N**: generic workflow engine, also with GenAI integration
- **Flyte** (https://flyte.org/): ML, AI, computation

Example with Apache Airflow

https://airflow.apache.org

Airflow overview

- Originally from Airbnb
- Features
  - dynamic, extensible, scalable workflows, programmable language-based workflows
    - workflows written as procedural code
- Good and easy to study to understand concepts of workflows:
  - suitable for data analytics/ETL, ML engineering and DevOps automation

Many service connectors to different capabilities for diverse types of tasks

- Airbyte
- Alibaba
- Amazon
- Apache Beam
- Apache Cassandra
- Apache Drill
- Apache Druid
- Apache HDFS
- Apache Hive
- Apache Kylin
- Apache Livy
- Apache Pig
- Apache Pinot
- Apache Spark
- Apache Sqoop
- Asana
- Celery
- IBM Cloudant
- Kubernetes
- Databricks
- Datadog
- DBT cloud
- Dingding
- Discord
- Docker
- Elasticsearch
- Exasol
- Facebook
- File Transfer Protocol (FTP)
- Github
- Google
- gRPC
- Hashicorp
- Hypertext Transfer Protocol (HTTP)
- Influx DB
- Internet Message Access Protocol (IMAP)
- Java Database Connectivity (JDBC)
- Jenkins
- Jira
- Microsoft Azure
- Microsoft PowerShell Remoting Protocol (PSRP)
- Microsoft SQL Server (MSSQL)
- Windows Remote Management (WinRM)
- MongoDB
- MySQL
- Neo4J
- ODBC
- OpenFaaS
- Opsgenie
- Oracle
- Pagerduty
- Papermill
- Plexus
- PostgreSQL
- Presto
- Qubole
- Redis
- Salesforce
- Samba
- Segment
- Sendgrid
- SFTP
- Singularity
- Slack
- Snowflake
- SQLite
- SSH
- Tableau
- Telegram
- Trino
- Vertica
- Yandex
- Zendesk

From https://airflow.apache.org/docs/

Cloud integration and big data support

- Several supports with known cloud providers
  - Microsoft Azure
  - Amazon Web Services
  - Databricks
  - Google Cloud Platform (Google Composer)
- Big data supports
  - Hadoop, Hive, Druid, Presto
- Distributed execution
  - Celery, Dask, and Kubernetes

Airflow workflow structure

- Workflow is a DAG (Direct Acyclic Graph)
  - a workflow consists of a set of activities/tasks represented in a DAG
  - workflow and activities are programed using Python
    - the workflow structures described in code
- Workflow activities are described by Airflow operator objects
  - tasks are created when instantiating operator objects

Airflow operators/tasks

- Tasks are implemented using operators
- Rich set of operators
  - we can program different kinds of tasks and integrate with different systems
- Different types of operators for workflow tasks
  - BashOperator, PythonOperator, EmailOperator, SimpleHttpOperator, BaseSQLOperator, BaseSensorOperator, DockerOperator, HiveOperator, SparkSubmitOperator,SageMakerTrainingOperator, PrestoToMysqlOperator, SlackAPIPostOperator
- Remember:
  - tasks implemented with such operators will be executed by corresponding services

Example for uploading state logs

```python
t_download_data =  PythonOperator(
    task_id="download_data",
    python_callable=download_data,
    op_kwargs={'source_file':source_file, 'dest_file':temp_dest_file},
    dag=dag,
    )
#the dest file from the download task will be used for analytics
t_basic_aggregration = PythonOperator(
    task_id='alarm_analytic',
    python_callable=basic_aggregation,
    op_kwargs={'input_file':temp_dest_file,'report_destination':report_destination}
    dag=dag,
    )
t_uploadgcs =  LocalFilesystemToGCSOperator(
    task_id="upload_local_file_to_gcs",
    src=report_destination,
    dst=gcs_dest_file,
    bucket=GCS_CONF["bucket"],
    gcp_conn_id=GCS_CONF["gcp_conn_id"],
    dag = dag
    )
```

```python
t_insert_data_warehouse = PythonOperator(
    task_id='insert_data_warehouse',
    python_callable=data_to_bigquery,
    op_kwargs={'input_data_src':f'file://{report_destination}',
               'table_id':BIGQUERY_CONF["table_id"],
               'project_id':BIGQUERY_CONF["project_id"],
               'credentials':credentials},
    dag=dag)


t_msnotification = PythonOperator(
    task_id='teams_notification',
    python_callable=post_notification,
    op_kwargs={'gcs_report':gcs_file_url,'teams_webhook':teams_webhook},
    dag=dag)


t_clean_data = PythonOperator(
    task_id='data_cleansing',
    python_callable=clean_data,
    op_kwargs={'dest_files':[temp_dest_file,report_destination]},
    dag=dag)
```

In our GIT course (tutorials)

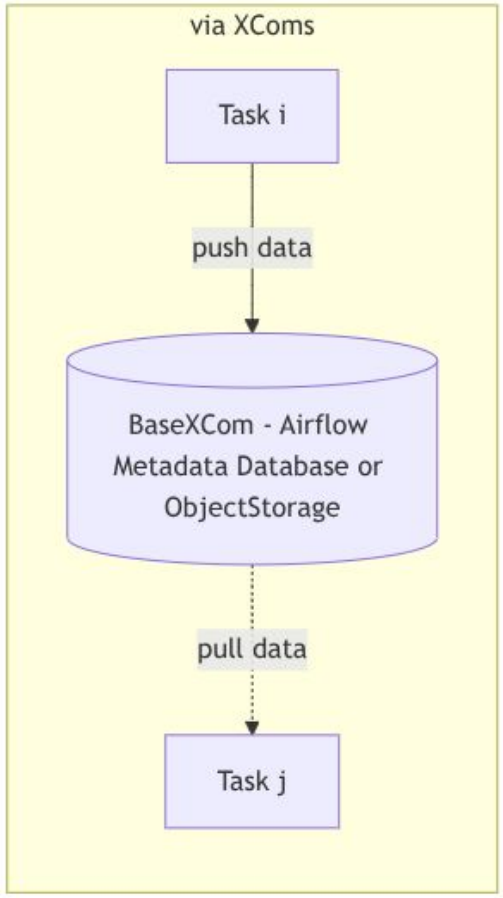
Example

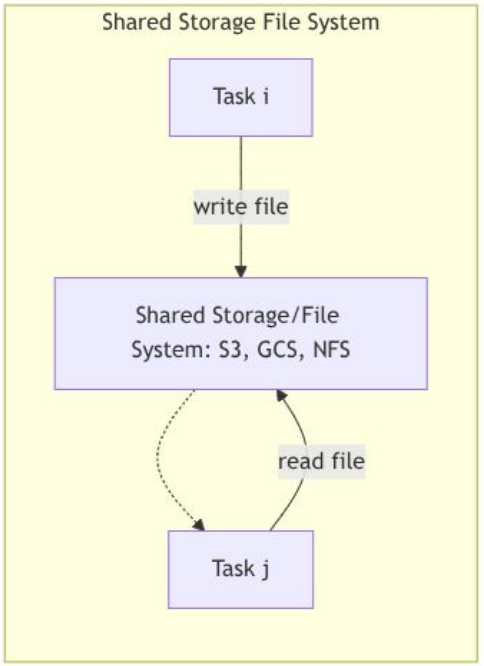
upstream task

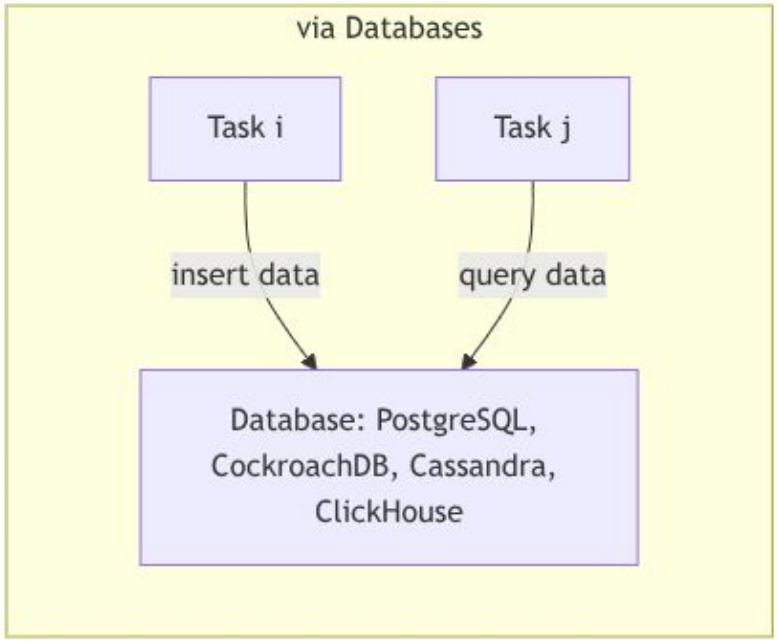
```
"""
the dependencies among tasks

now you have to remember how different tasks exchange data:
- they pass data via files and you use a local file system, but
task A and task B are not executed in the same machine
- they pass data via a global data storage, then some upload/download of data
must be implemented.

thus, you have to see the task implementation in detail. This example, basically,
works only for local or file sharing systems as we implement download, check quality,
clean data, etc. using local file systems.
"""

t_download_data >> t_basic_aggregration >> t_uploadgcs >> t_insert_data_warehouse
t_insert_data_warehouse >> t_msnotification  >> t_clean_data
```

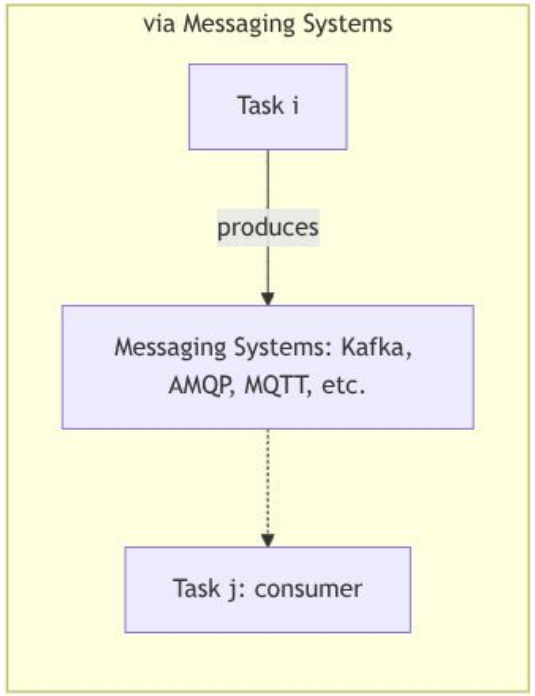
downstream task

Data exchange among tasks: possibilities







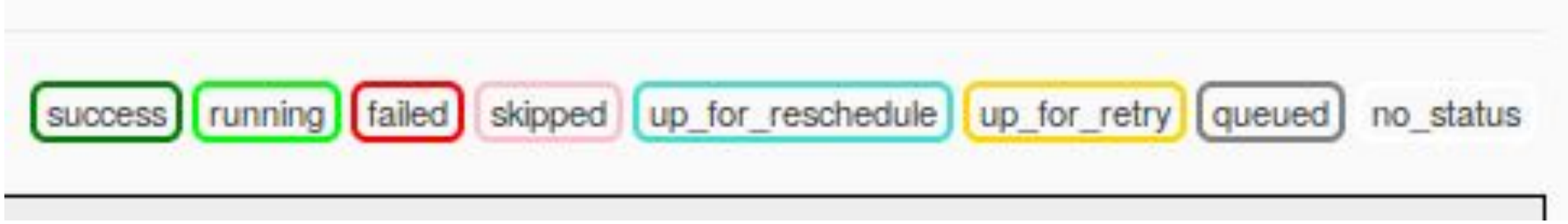


Scheduling and execution

- You can schedule the workflow like a cron job
  - execute once, every minutes, hours, …
- Trigger from external
  - tasks can be triggered as normal (upstream tasks finishes, dependencies)
  - or specific triggers
- Very suitable ingestion and batch analytics job managements
  - the ingestion and analytics are done within tasks
  - schedule based on analytics needs

Task lifecycle

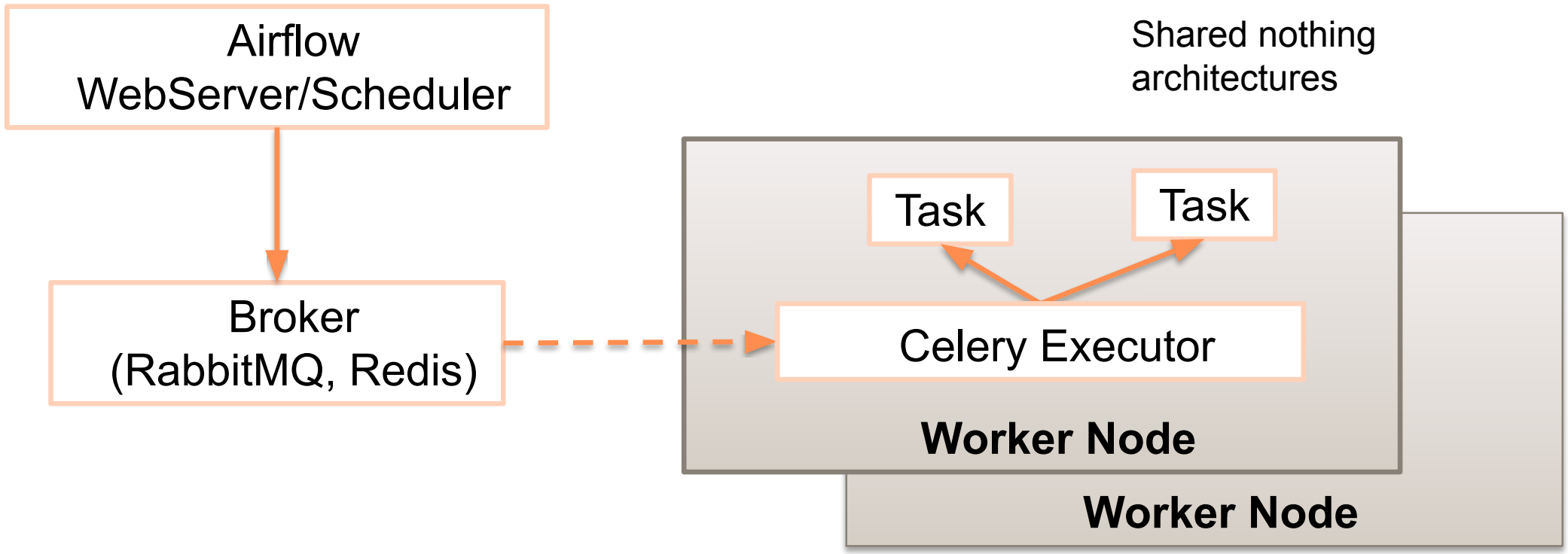
Different states



- Performance metrics can be determined based on states and structures
- Interesting in performance analytics?
  - Check https://doi.org/10.1016/j.future.2007.01.003

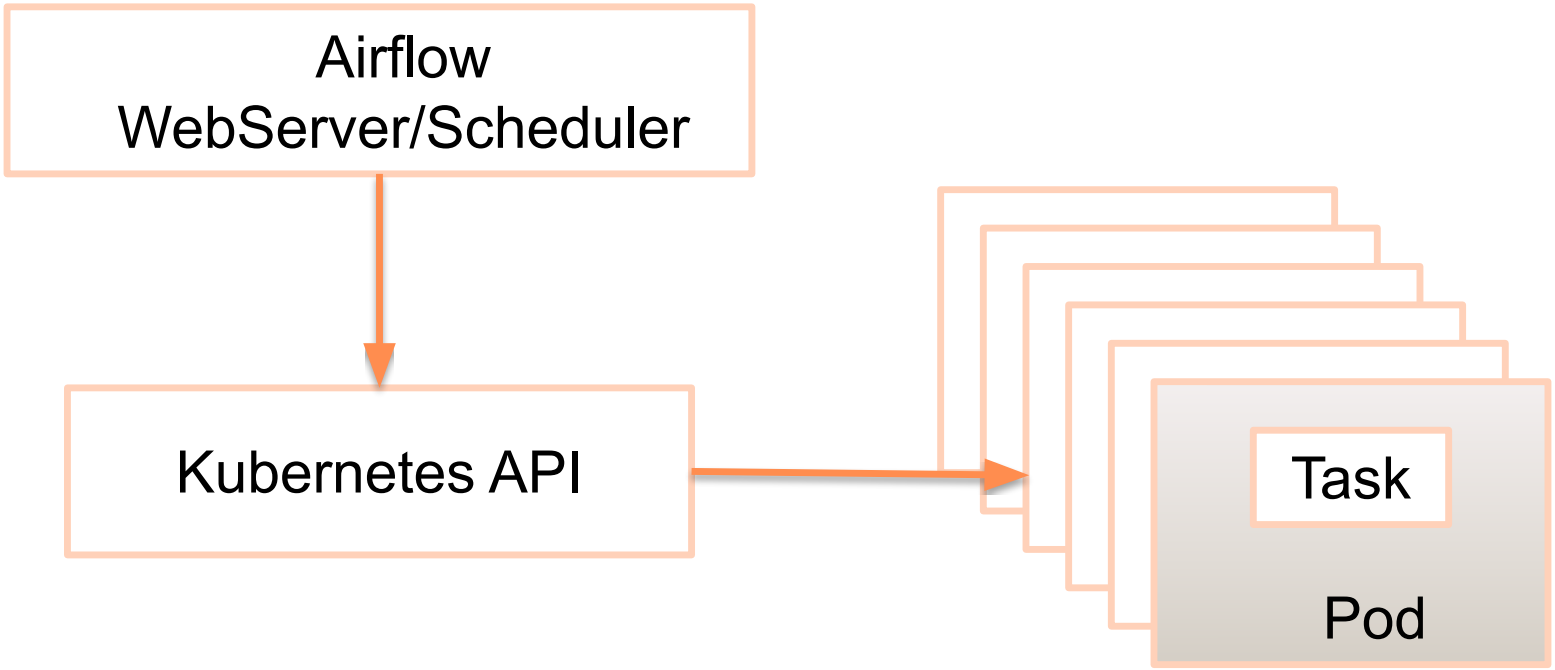
Distributed tasks (1)

You can scale Airflow using workers deployed in different nodes managed by Celery (http://www.celeryproject.org)



Distributed tasks (2)

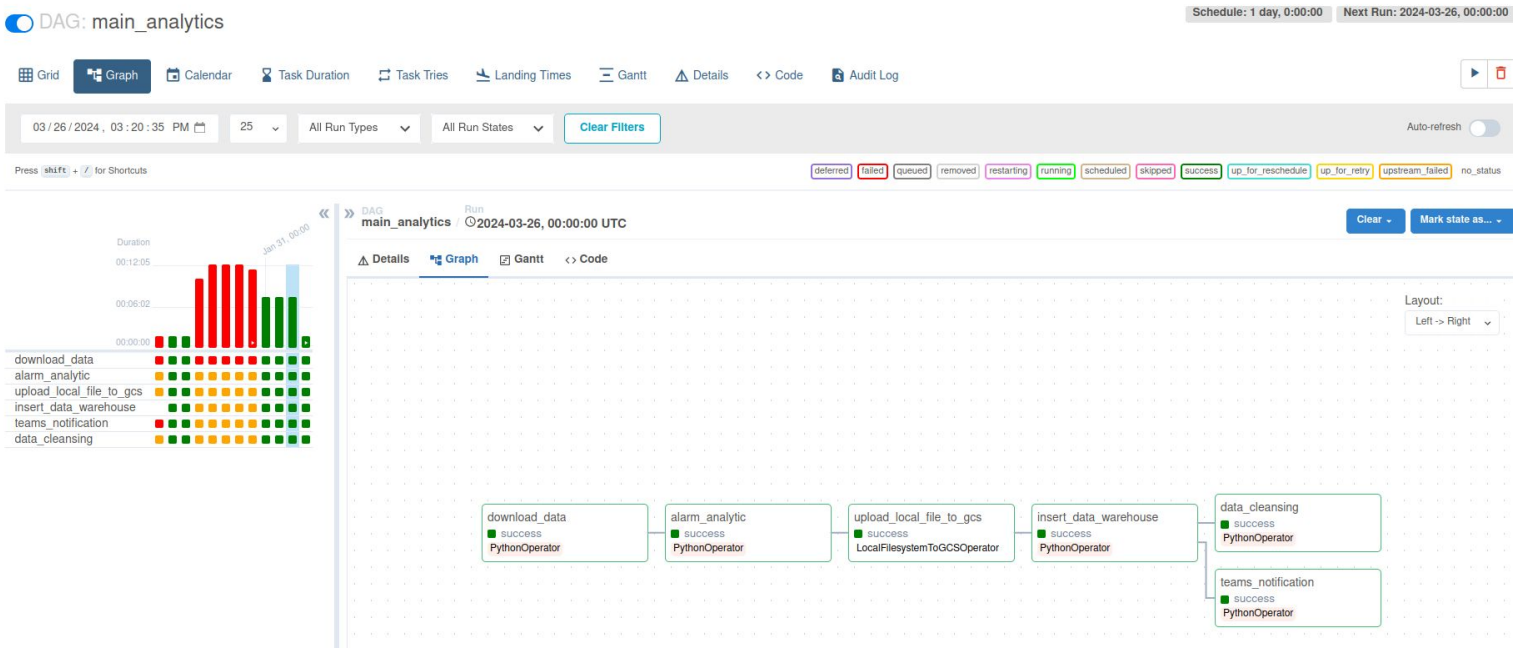
You can scale Airflow to run tasks in Kubernetes



Google Cloud Composer: use Kubernetes

Monitoring UI

Download data from various services (via pre-defined API/endpoints) ⇒ analyze data ⇒ store results to Google storage and BigQuery ⇒ send notifications about the result to Teams



Summary

- Focus:
  - practical programming with common workflow systems:
    - Apache Airflow: for data analytics and platform management
    - Serverless workflows using function-as-a-service: e.g., AWS Steps
- Action:
  - hands-on and work on concrete examples
    - try to see if you can implement previous use cases/scenarios in your work with workflows
  - offering workflows as a service in your platform!
    - suggest to do some hands-on by configuring and deploying Airflow

Thanks!

Hong-Linh Truong
Department of Computer Science

rdsea.github.io

A!

—

Kiitos
aalto.fi